

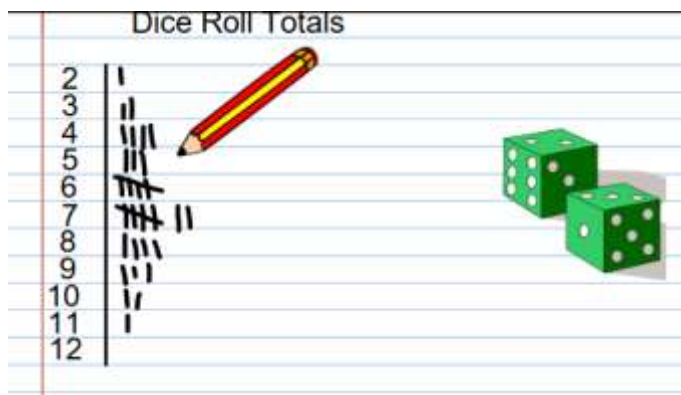
2212 Java Software Dev I

Midterm Program Project – Dice Roller

Project Setting

Imagine your statistics professor gives you an assignment: roll two six-sided dice 1,000 times and record how often each possible sum—ranging from 2 to 12—appears.

How would you tackle it? You'd likely start by writing the numbers 2 through 12 down the side of a sheet of paper. Then, for each roll, you'd add the two dice together and mark a tally next to the corresponding sum. After completing all 1,000 rolls, you'd total the tallies for each sum to see their frequencies.



While this task is tedious for humans, it's effortless for computers. So instead of rummaging through the game cupboard for dice, let's write a program to simulate the process.

Project Design

We want to build a flexible dice-rolling simulation. Instead of assuming we're always rolling two six-sided dice, the program should ask the user:

- How many dice to roll
- How many sides does each die have
- How many times to roll the dice

This way, the simulation can handle a wide range of dice configurations.

Rolling Computer Dice

How will we simulate the rolling of dice in code? How do we simulate random events in code? With a random number generator. It is important to note that we must simulate the rolling of each die individually. To illustrate why, imagine we are rolling two six-sided dice and we do that by choosing a random number from 2 to 12. What are the odds that each number is rolled? One chance in 11. Does that match the odds of rolling two six-sided dice? It does not.

If someone is going to roll two six-sided dice and they ask you to bet on the result, if you understand the odds, then you will bet on 7. Why? Because more combinations of two six-sided dice make 7 than the other possible results. So, to accurately simulate rolling x number of dice, we must roll each die individually and sum the rolls to get the total.

Another thought about simulating the roll of a die. Humans number the sides of a standard die from 1 to 6, but Java likes things to be zero-based. Therefore, your code will be cleaner if you roll zero-based dice. The user will never know the difference.

Project Requirements

Our solution will use 3 classes; `Main`, `DiceFactory`, and `DiceManager`. Three `.java` files will be provided to help you get started in the right direction. **No changes should be made to `Main.java`.**

Here are the specifications for the `DiceFactory` class:

- A `DiceFactory` needs to keep track of how many times each sum of the dice is rolled. We need an organized collection of numbers. We can calculate the smallest and largest possible results based on the number of dice and number of sides. Therefore, we can calculate **exactly** how large the collection should be. (Don't waste memory.)

AI Tip: Research how to calculate the exact dimensions of the array.

- A `DiceFactory` needs a behavior that runs a dice-rolling simulation. This behavior needs to know how many times to roll the dice, how many dice are to be rolled each time, and how many sides are on each die. The method should run the simulation and store the results. The method does not need to return anything.
- A `DiceFactory` needs a behavior that returns the results of the last simulation as a `String`. This behavior should build and return a `String` that includes the sum of each of the numbers. The behavior does not need to receive any information since it has access to the instance variables that describe the last simulation and the results. If this method is called before a simulation has been run, return an empty `String`.

For example, if we roll two six-sided dice 1000 times, then the return string would look something like this when displayed on the console:

```
2)      4
3)     10
4)     21
...     ...
12)     2
```

- So that the results can be analyzed in different ways, a `DiceFactory` needs a behavior that returns the results of the last simulation as an `int` array. This is a bit tricky. If we simply return the results array variable, we will actually return the memory address to the array. This is called a “shallow copy”. Returning the address of the array would grant other programs direct access to the results array inside the `DiceFactory` object. That violates the rules of encapsulation. What we need is a “deep copy”. So, we need to return a clone of the array. *AI Tip*: Research how to clone an array in Java. What if someone calls this method before running a simulation? The results array will be `null` and the clone operation will fail. So, we need to guard against this case. If this method is called before running a simulation, return `null`.

Here are the specifications for the `DiceManager` class:

- A `DiceManager` needs a `DiceFactory` to simulate rolling dice and a `Scanner` for user input. The constructor should instantiate both objects.
- The “go” method should display a menu of choices, get user input, and display the results. Note that one `DiceFactory` object should be used throughout the program run. Do not instantiate a new `DiceFactory` object each time a simulation is run. This process should continue until the user selects the “Quit” option. To get a number from the user, use the provided helper method `inputInt`. This method has a parameter for the maximum number allowed. *AI Tip*: Inspect the `Integer` class for a constant that is useful in the cases where we don’t care about the maximum value.

Program Execution Examples

```
Welcome to Dice Roller.

Choose an option -
0) Quit
1) Quick roll with the default values (rolls = 1000, dice = 2, sides = 6)
2) Custom dice rolling (You enter the number of rolls, dice and sides.)
3) Get the array showing the results of the last simulation.

Enter your choice: foo
Make sure you enter an integer. Try again.

Enter your choice: 4
That number is out of range [0 to 3]. Try again.
```

Invalid input should be caught and handled gracefully. This logic is provided for you in the `inputInt` method, so use it every time you need the user to enter an `int`.

Choose an option -

- 0) Quit
- 1) Quick roll with the default values (rolls = 1000, dice = 2, sides = 6)
- 2) Custom dice rolling (You enter the number of rolls, dice and sides.)
- 3) Get the array showing the results of the last simulation.

Enter your choice: 3

Results array : null

A simulation hasn't been run, so the results array is null.

Choose an option -

- 0) Quit
- 1) Quick roll with the default values (rolls = 1000, dice = 2, sides = 6)
- 2) Custom dice rolling (You enter the number of rolls, dice and sides.)
- 3) Get the array showing the results of the last simulation.

Enter your choice: 1

2) 29
3) 48
4) 98
5) 105
6) 130
7) 177
8) 133
9) 118
10) 83
11) 57
12) 22

Quick roll: two 6-sided dice rolled 1000 times. Note how the numbers line up.

Choose an option -

- 0) Quit
- 1) Quick roll with the default values (rolls = 1000, dice = 2, sides = 6)
- 2) Custom dice rolling (You enter the number of rolls, dice and sides.)
- 3) Get the array showing the results of the last simulation.

Enter your choice: 3

Results array : [29, 48, 98, 105, 130, 177, 133, 118, 83, 57, 22]

Research the Arrays.toString method.

```
Choose an option -
0) Quit
1) Quick roll with the default values (rolls = 1000, dice = 2, sides = 6)
2) Custom dice rolling (You enter the number of rolls, dice and sides.)
3) Get the array showing the results of the last simulation.
```

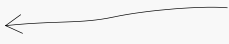
```
Enter your choice: 2
```

```
Enter the number of times you want to roll the dice: 10000
```

```
Enter the number of sides on each die: 8
```

```
Enter the number of dice you want to roll each time: 3
```

```
3)    16
4)    50
5)   124
6)   220
7)   298
8)   433
9)   557
10)  718
11)  813
12)  870
13)  954
14)  910
15)  967
16)  783
17)  673
18)  554
19)  394
20)  297
21)  197
22)  102
23)   56
24)   14
```



Custom roll. Note the range of possible results and the uneven distribution.

Expectations

The program is expected to compile successfully, include identification comments at the top, and produce output that exemplifies good grammar and spelling. Failure to meet these standards may result in point deductions.